# PROJECT_PLAN — OmniMarket · Operations Hub

> Estado del documento: **Plan preliminar. Todavía no se ha implementado nada.** Todos los requisitos de la matriz están en estado `Pendiente`. Fuente principal: *Especificación Técnica Oficial — Proyecto 2: OmniMarket — Operations Hub* (4 páginas). Las referencias `§n` remiten a las secciones de ese documento.

---

## 1. Objetivo

Construir un dashboard web de analítica y administración de e-commerce que:

1. Cumpla **todos** los requisitos obligatorios de la especificación oficial (§1 a §6).
2. Sea **verificable por el evaluador** en menos de 10 minutos con su lista de comprobación (§6):
    1. Entrar a la URL pública y autenticarse con OAuth.
    2. Medir la latencia de `GET /api/test/analytics`.
    3. Arrastrar una tarjeta del Kanban y comprobar que persiste.
    4. Revisar que el pipeline de GitHub Actions está en verde.
3. Tenga calidad profesional de interfaz, sin inflar la arquitectura: un solo proyecto Next.js, una base de datos y un despliegue.

**Criterio de dictamen (§6):** el proyecto solo se aprueba si está desplegado y accesible en producción, con el CI/CD activo y con los endpoints de auditoría respondiendo bien. Por eso el plan **despliega desde la Fase 0** y construye los endpoints de prueba antes que la interfaz.

### 1.1 Pesos de evaluación (§1)

La tabla del PDF aparece desalineada al extraer el texto. Esta es la reconstrucción más coherente; conviene confirmarla con el docente:

| # | Módulo | Peso | Criterio de aprobación | Métrica de validación |
|---|---|---|---|---|
| 1 | Arquitectura & Next.js 14+ | 25 % | App Router, TypeScript y Tailwind | Renderizado sin errores de consola |
| 2 | Endpoints de auditoría API | 25 % | Endpoints `/api/test/*` habilitados | Respuesta paginada < 300 ms |
| 3 | Drag & Drop y CRUD avanzado | 20 % | Kanban interactivo y Bulk Update | Persistencia optimista inmediata |
| 4 | Pruebas automatizadas & CI/CD | 15 % | 10 tests unitarios + 3 flujos E2E + Actions | Pipeline de GitHub Actions en verde |
| 5 | Despliegue en producción | 15 % | URL pública activa en Vercel/Railway | Disponibilidad y SSL válido |

> **Nota (§5):** si no hay suite de pruebas, la nota del módulo de Garantía de Calidad es **0.0** de forma automática.

## 2. Requisitos funcionales

Identificadores: **RF-xx**. Prioridad: **O** = obligatorio (aparece en la especificación), **D** = derivado (no está escrito, pero hace falta para cumplir uno obligatorio), **M** = mejora opcional.

### M1 · Autenticación (§3, §6)

| ID | Requisito | Prioridad |
|---|---|---|
| RF-01 | Inicio de sesión con **Google** (OAuth 2.0). | O |
| RF-02 | Inicio de sesión con **GitHub** (OAuth 2.0). | O |
| RF-03 | La autenticación es **exclusivamente OAuth**: no hay usuario y contraseña en producción ("Autenticación exclusiva", §3). | O |
| RF-04 | Las rutas privadas redirigen a `/login` si no hay sesión (lo valida el E2E 1, §5). | O |
| RF-05 | Cierre de sesión y menú de usuario con avatar y nombre. | D |
| RF-06 | Roles (ADMIN / OPERATOR / VIEWER) con permisos por acción. | M |

### M2 · Dashboard analítico (Resumen, §3, §4)

| ID | Requisito | Prioridad |
|---|---|---|
| RF-10 | KPIs de ventas: ingresos, número de pedidos, **ticket promedio** y **margen** (§5 los menciona como cálculos financieros). | O |
| RF-11 | Consultas de métricas y de paginación con respuesta del backend **< 300 ms** (§3). | O |
| RF-12 | Skeletons mientras cargan los datos, no spinners genéricos (§4). | O |
| RF-13 | Gráficas: ventas por día, ventas por categoría y productos más vendidos. | D |
| RF-14 | Filtro por rango de fechas que alimenta los KPIs y las gráficas. | D |
| RF-15 | Comparativa contra el periodo anterior (variación %). | M |

### M3 · CRUD de productos (§1, §5)

| ID | Requisito | Prioridad |
|---|---|---|
| RF-20 | Crear, listar, editar y eliminar productos. | O |
| RF-21 | Tabla paginada del lado del servidor (< 300 ms), con búsqueda y filtros (categoría, estado, stock). | O |
| RF-22 | Validación con esquemas Zod, compartidos entre cliente y servidor (§5). | O |
| RF-23 | El producto recién creado aparece en la tabla (E2E 2, §5). | O |

| ID | Requisito | Prioridad |
|---|---|---|
| RF-24 | Confirmación modal antes de eliminar. | D |
| RF-25 | Imagen del producto, exportar a CSV. | M |

## M4 · CRUD de pedidos (§1, §5)

| ID | Requisito | Prioridad |
|---|---|---|
| RF-30 | Crear, listar, ver detalle, editar y cancelar/eliminar pedidos. | O |
| RF-31 | Un pedido tiene cliente, líneas (producto, cantidad, precio y costo unitarios), total y estado. | D |
| RF-32 | Tabla paginada del lado del servidor, con filtros por estado, fecha y cliente. | O |
| RF-33 | El total se calcula **en el servidor** a partir de las líneas; no se acepta el que mande el cliente. | D |
| RF-34 | Historial de cambios de estado del pedido. | M |

## M5 · Kanban Drag & Drop (§1, §3, §4, §6)

| ID | Requisito | Prioridad |
|---|---|---|
| RF-40 | Tablero de **pedidos** con una columna por estado: Pendiente → En proceso → Enviado → Entregado (+ Cancelado). | O |
| RF-41 | Arrastrar una tarjeta a otra columna cambia el estado del pedido. | O |
| RF-42 | **Persistencia optimista:** la tarjeta se mueve al instante y la petición se envía en segundo plano. | O |
| RF-43 | Si el servidor falla, la tarjeta **vuelve a su columna original** y se muestra un toast de error. | O |
| RF-44 | Si el servidor responde bien, se muestra un toast o modal de éxito. | O |
| RF-45 | El cambio persiste al recargar y se puede consultar en `GET /api/test/kanban-status`. | O |
| RF-46 | Accesibilidad: también se puede mover con teclado. | D (dnd-kit lo trae) |
| RF-47 | Reordenar dentro de una misma columna. | M |

## M6 · Bulk Update (§1, §3, §4)

| ID | Requisito | Prioridad |
|---|---|---|
| RF-50 | Seleccionar varias filas (productos y pedidos) y actualizarlas a la vez. | O |
| RF-51 | Productos: cambiar estado, categoría, stock o precio (ajuste en %). Pedidos: cambiar estado. | D |
| RF-52 | Actualización optimista con reversión si hay error, y toast de confirmación (§4). | O |
| RF-53 | La operación es atómica: se aplica a todas las filas o a ninguna (transacción). | D |
| RF-54 | Límite de filas por lote (p. ej. 100) validado con Zod. | D |

## M7 · Endpoints de auditoría (§2)

| ID | Requisito | Prioridad |
|---|---|---|
| RF-60 | `GET /api/test/analytics?page=1&limit=50` devuelve métricas de ventas paginadas en **< 300 ms**. | O |
| RF-61 | `POST /api/test/reset-data` restablece la base de datos a un conjunto de prueba conocido y determinista. | O |
| RF-62 | `GET /api/test/kanban-status` devuelve el estado actualizado de un pedido después de moverlo. | O |
| RF-63 | Respuestas JSON con forma estable, códigos HTTP correctos, errores 400 ante parámetros inválidos y cabecera `Server-Timing`. | D |

**Contratos propuestos (preliminares):**

```
GET /api/test/analytics?page=1&limit=50[&from=YYYY-MM-DD&to=YYYY-MM-DD]
200 {
  "summary":    { "revenue", "orders", "avgTicket", "grossMargin", "grossMarginPct", "currency": "MXN"
},
  "data":       [ { "date", "orders", "revenue", "cost", "avgTicket", "margin" } ],
  "pagination": { "page", "limit", "total", "totalPages" },
  "meta":       { "durationMs", "generatedAt" }
}
400 { "error": "INVALID_QUERY", "issues": [...] }   // p. ej. limit > 100

POST /api/test/reset-data
200 { "ok": true, "seed": "omnimarket-v1", "counts": { "products", "customers", "orders" },
"durationMs" }

GET /api/test/kanban-status?orderId=ORD-0001
200 { "orderId", "status", "updatedAt", "lastMove": { "from", "to", "at", "by" } }
GET /api/test/kanban-status          // sin parámetros: conteo por columna
200 { "columns": { "PENDING": 12, "PROCESSING": 8, ... }, "updatedAt" }
404 { "error": "ORDER_NOT_FOUND" }
```

> El PDF no dice qué se pagina en `/analytics`. Propuesta: `data` = métricas de ventas **agrupadas por día** (paginables) y `summary` = KPIs del periodo. Así se cumplen "métricas de ventas" y "paginación" al mismo tiempo.

## M8 · Experiencia de usuario transversal (§4)

| ID | Requisito | Prioridad |
|---|---|---|
| RF-70 | Skeletons en **toda** consulta y mutación asíncrona (tablas, KPIs, gráficas, Kanban, formularios al guardar). | O |
| RF-71 | Toast o modal en todo cambio de estado del Kanban y en toda edición en lote. | O |
| RF-72 | Reversión visible del cambio optimista ante error. | O |
| RF-73 | Cero errores en la consola del navegador en todas las pantallas. | O |
| RF-74 | Diseño responsive, modo oscuro, estados vacíos y páginas de error amigables. | M |

---

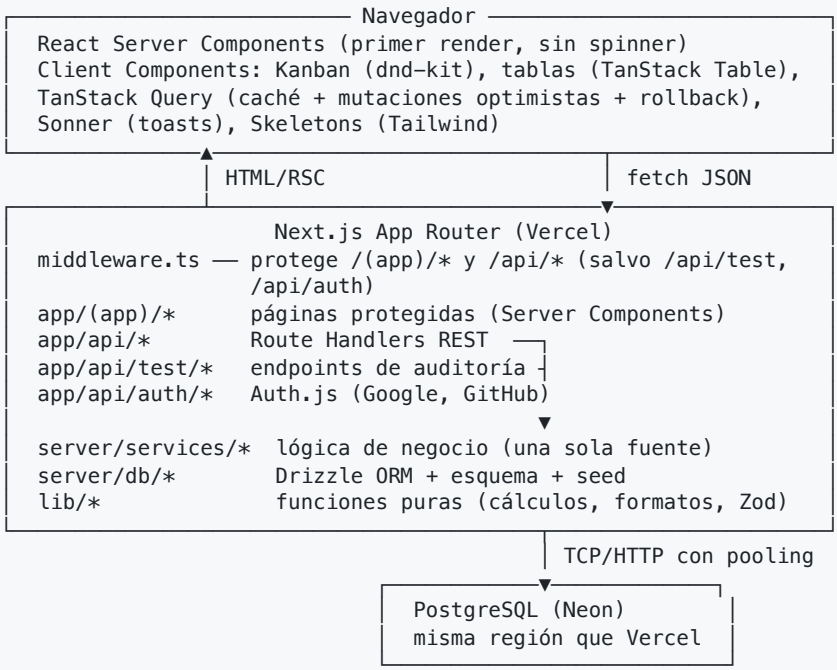
# 3. Requisitos técnicos

| ID | Requisito | Fuente | Prioridad |
|---|---|---|---|
| RT-01 | Next.js ≥ **14** con **App Router** (sin `pages/`). Se usará la última versión estable al iniciar, fijada en `package.json`. | §1 | O |
| RT-02 | **TypeScript** en modo `strict`. | §1 | O |
| RT-03 | **Tailwind CSS** para todos los estilos. | §1, §4 | O |
| RT-04 | Latencia del backend < 300 ms en métricas y paginación. | §2, §3 | O |
| RT-05 | OAuth 2.0 con Google y GitHub. | §3 | O |
| RT-06 | ≥ 10 pruebas unitarias o de integración con Vitest o Jest, que cubran: cálculos financieros (ticket promedio, márgenes), formateo de moneda, esquemas Zod y lógica de filtrado. | §5 | O |
| RT-07 | ≥ 3 flujos E2E con Playwright o Cypress: autenticación, creación y Drag & Drop. | §5 | O |
| RT-08 | Archivo `.github/workflows/ci.yml` que **impida desplegar en Vercel si fallan los tests**. | §5 | O |
| RT-09 | Despliegue público en Vercel o Railway con SSL válido. | §1, §6 | O |
| RT-10 | Sin errores de consola. | §1 | O |
| RT-11 | Datos de prueba deterministas y reiniciables (los necesita RF-61 y los E2E repetibles). | §2 | D |
| RT-12 | Secretos solo en variables de entorno; `.env.example` versionado y `.env*` ignorados. | — | D |
| RT-13 | ESLint, Prettier y `tsc --noEmit` en el CI. | — | D |

# 4. Arquitectura

## 4.1 Principio rector

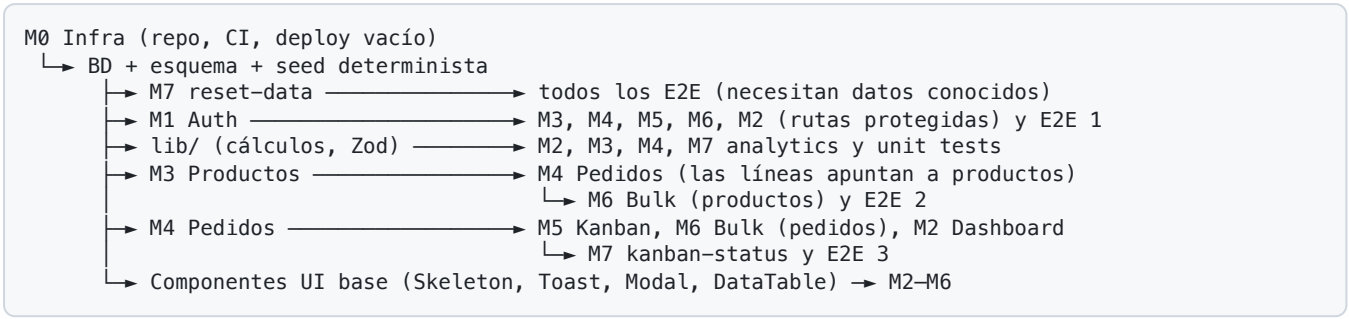
**Un monolito Next.js** que sirve interfaz y API, con **una base de datos PostgreSQL gestionada** y **un solo despliegue en Vercel**. No hay microservicios, ni backend aparte, ni colas, ni Redis. Cada capa existe porque la pide un requisito.




## 4.2 Decisiones clave

| Decisión | Elección | Motivo |
|---|---|---|
| API | **Route Handlers REST** (`app/api/...`) consumidos con TanStack Query. Los Server Components leen directo de `server/services`. | Los endpoints `/api/test/*` ya tienen que ser REST; con TanStack Query, el patrón optimista con `onMutate` / `onError` (rollback) / `onSettled` es explícito y fácil de probar. Los Server Actions serían una segunda forma de mutar sin aportar nada. |
| Lógica de negocio | Capa `server/services` compartida por páginas, API REST y endpoints de prueba. | `/api/test/analytics` y el dashboard usan **el mismo** cálculo: lo que mide el evaluador es lo que ve el usuario. |
| Cálculos | Funciones puras en `lib/` (ticket promedio, margen, formato de moneda, filtros). | Son justo lo que §5 pide probar, sin base de datos ni mocks. |
| Validación | Zod en el borde (entrada de cada Route Handler y formularios con react-hook-form). | Lo pide §5 y da tipos gratis. |
| Sesión | Auth.js con **estrategia JWT** y adaptador de BD para usuarios y cuentas. | El middleware valida la sesión sin consultar la BD (rápido) y el E2E puede "simular sesión" firmando un JWT de prueba (§5). |
| Analítica rápida | Índices + tabla agregada `daily_sales` que se recalcula al escribir pedidos y en el seed. | Garantiza < 300 ms aunque crezca el número de pedidos: la consulta paginada lee como mucho 100 filas ya agregadas. |
| Despliegue | El CI despliega con Vercel CLI **solo si** pasan lint, typecheck, unit, build y E2E. | Cumple de forma literal y verificable "bloquear despliegues en Vercel ante fallos de tests" (§5). |

### 4.3 Dependencias entre módulos




**Ruta crítica:** Infra → BD/seed → reset-data → Auth → Pedidos → Kanban → E2E → deploy bloqueado por CI.

---

## 5. Tecnologías recomendadas

| Necesidad | Librería | Por qué esta y no otra |
|---|---|---|
| Framework | **Next.js** (última estable ≥ 14), App Router, React 19 | Lo exige §1. |
| Lenguaje | **TypeScript** `strict` | Lo exige §1. |
| Estilos | **Tailwind CSS** | Lo exige §1. |
| Componentes base | **shadcn/ui** (Radix + Tailwind): Dialog, DropdownMenu, Checkbox, Select, Skeleton | El código queda en el repo (sin dependencia opaca), es accesible y está hecho con Tailwind. |
| Iconos | lucide-react | Ligero; es el que usa shadcn. |
| Base de datos | **PostgreSQL en Neon** (región `us-east-1` , igual que la función de Vercel `iad1` ) | Agregaciones SQL potentes, plan gratuito, *pooling* para serverless, ramas para pruebas. Ver §6.1. |
| ORM / migraciones | **Drizzle ORM** + drizzle-kit | Arranque en frío más ligero que Prisma (importa para los 300 ms en serverless), SQL tipado y transparente, adaptador oficial de Auth.js. |
| Autenticación | **Auth.js (next-auth v5)** con proveedores Google y GitHub + `@auth/drizzle-adapter` | Estándar de facto en Next.js, OAuth 2.0 resuelto y JWT compatible con el middleware. |
| Estado del servidor en cliente | **TanStack Query v5** | Caché, invalidación y mutaciones optimistas con rollback documentado. |
| Tablas | **TanStack Table v8** | Selección de filas (Bulk Update), paginación y ordenamiento manual del lado del servidor. Sin estilos propios: se combina con Tailwind. |
| Drag & Drop | **@dnd-kit/core** + @dnd-kit/sortable | Mantenido, accesible (teclado y lectores de pantalla) y compatible con React 19. `react-beautiful-dnd` está deprecado. |
| Formularios | react-hook-form + @hookform/resolvers/zod | Reutiliza los esquemas Zod del servidor. |
| Validación | **Zod** | Lo nombra §5. |

| Necesidad | Librería | Por qué esta y no otra |
|---|---|---|
| Gráficas | **Recharts** | Declarativa y suficiente para barras, líneas y área. Se renderiza solo en cliente para evitar avisos de SSR. |
| Toasts | **Sonner** | Pequeña y con API de promesas ( `toast.promise` ) ideal para mutaciones. |
| Fechas | date-fns (+ `Intl` para moneda) | Sin dependencias pesadas. Moneda con `Intl.NumberFormat('es-MX', { currency: 'MXN' })` . |
| Datos de prueba | @faker-js/faker con **semilla fija** | El seed determinista que pide RF-61. |
| Unit / integración | **Vitest** + @testing-library/react + jsdom | Rápido y compatible con TS/ESM sin configuración extra. |
| E2E | **Playwright** | Drag & Drop fiable, *traces* en CI y escucha de `console` para RF-73. |
| Calidad | ESLint ( `next/core-web-vitals` , `typescript-eslint` ), Prettier | Se revisa en CI. |
| CI/CD | GitHub Actions + Vercel CLI | Lo exige §5. |

**Lo que se descarta a propósito:** Redux/Zustand (TanStack Query basta), tRPC/GraphQL (REST ya lo exigen los endpoints de prueba), Redis o colas (la tabla agregada resuelve la latencia) y microservicios.

---

## 6. Base de datos

### 6.1 Recomendación: PostgreSQL gestionado (Neon)

- **Por qué SQL relacional:** pedidos → líneas → productos → clientes es relacional por naturaleza, y la analítica (sumas, márgenes, agrupación por día y categoría) es justo para lo que sirve SQL.
- **Por qué Postgres y no MySQL** (el equipo ya usó MySQL en SIFAT): mejor integración con Vercel y Neon, `TRUNCATE ... RESTART IDENTITY CASCADE` para un reset rápido, enums nativos y funciones de ventana. MySQL en Railway también serviría; si el equipo prefiere MySQL, el diseño no cambia.
- **Por qué Neon:** *serverless* con *pooling* integrado, misma región que Vercel y una **rama aparte** para E2E/preview, de modo que `reset-data` nunca toca datos que alguien esté viendo.
- **Alternativa equivalente:** Supabase Postgres o Railway Postgres.

## 6.2 Modelo de datos preliminar

```
users               (Auth.js)  id, name, email, image, role[ADMIN|OPERATOR|VIEWER] = OPERATOR,
created_at
accounts            (Auth.js)  user_id → users, provider[google|github], provider_account_id, ...
                    -- Con JWT no hace falta la tabla sessions.

categories          id, name (unique), slug
products            id (uuid), sku (unique), name, description, category_id → categories,
                    price_cents INT, cost_cents INT, stock INT ≥ 0,
                    status[ACTIVE|DRAFT|ARCHIVED], created_at, updated_at
                    idx: (status), (category_id), (name trgm opcional), (updated_at desc)

customers           id, name, email (unique), city, created_at

orders              id (uuid), code ("ORD-0001", unique, legible para el evaluador),
                    customer_id → customers,
                    status[PENDING|PROCESSING|SHIPPED|DELIVERED|CANCELLED],
                    subtotal_cents, tax_cents, total_cents, cost_cents,
                    version INT (control de concurrencia optimista),
                    placed_at, updated_at
                    idx: (status, updated_at desc), (placed_at), (customer_id)

order_items         id, order_id → orders ON DELETE CASCADE, product_id → products,
                    quantity INT > 0, unit_price_cents, unit_cost_cents

order_status_events id, order_id → orders, from_status, to_status, actor_user_id → users NULL,
                    source[KANBAN|BULK|FORM|SEED], created_at
                    -- alimenta "lastMove" en /api/test/kanban-status

daily_sales         date (PK), orders_count, revenue_cents, cost_cents
                    -- agregado; se recalcula en la misma transacción que crea, edita o cancela pedidos
```

**Convenciones:**

- El dinero se guarda en **centavos (enteros)** para evitar errores de punto flotante. Se formatea solo al mostrarlo.
- `version` en `orders`: el PATCH envía la versión esperada; si no coincide, responde 409 y el cliente revierte. Esto cubre el caso de dos personas moviendo la misma tarjeta.
- **Seed `omnimarket-v1`:** semilla fija de faker, aproximadamente 8 categorías, 120 productos, 60 clientes y 1 500 pedidos repartidos en 180 días. Pedidos ancla con código fijo ( `ORD-0001` … `ORD-0010` ) en estados conocidos para que los E2E y el evaluador tengan referencias estables.
- **Margen bruto** = (ingresos − costo) / ingresos. **Ticket promedio** = ingresos / pedidos, excluyendo los cancelados. Se definen una sola vez en `lib/metrics.ts` .

---

## 7. Estructura de carpetas propuesta

```
omnimarket-operations-hub/
├─ .github/workflows/ci.yml          # lint → typecheck → unit → build → e2e → deploy (solo main)
├─ drizzle/                          # migraciones SQL generadas
├─ e2e/
│  ├─ fixtures.ts                    # sesión simulada + falla si hay console.error / pageerror
│  ├─ auth.spec.ts                   # E2E 1
│  ├─ create-product.spec.ts         # E2E 2
│  └─ kanban-dnd.spec.ts             # E2E 3
├─ public/                           # favicon, logo (evita 404 en consola)
├─ src/
│  ├─ app/
│  │  ├─ (auth)/login/page.tsx
│  │  ├─ (app)/layout.tsx            # shell: sidebar, topbar, QueryProvider, Toaster
│  │  ├─ (app)/dashboard/page.tsx  + loading.tsx (skeleton)
│  │  ├─ (app)/products/page.tsx   + loading.tsx, [id]/, new/
│  │  ├─ (app)/orders/page.tsx     + loading.tsx, [id]/, new/
│  │  ├─ (app)/kanban/page.tsx     + loading.tsx
│  │  ├─ api/auth/[...nextauth]/route.ts
│  │  ├─ api/products/route.ts       # GET (paginado) · POST
│  │  ├─ api/products/[id]/route.ts  # GET · PATCH · DELETE
│  │  ├─ api/products/bulk/route.ts  # PATCH
│  │  ├─ api/orders/...              # mismo patrón + [id]/status (PATCH, usado por el Kanban)
│  │  ├─ api/orders/bulk/route.ts
│  │  ├─ api/test/analytics/route.ts
│  │  ├─ api/test/reset-data/route.ts
│  │  ├─ api/test/kanban-status/route.ts
│  │  ├─ error.tsx · not-found.tsx · layout.tsx · globals.css
│  ├─ components/
│  │  ├─ ui/                         # shadcn: button, dialog, skeleton, checkbox...
│  │  ├─ data-table/                 # DataTable genérica + barra de Bulk Update
│  │  ├─ kanban/                     # Board, Column, Card (dnd-kit)
│  │  ├─ dashboard/                  # KpiCard, SalesChart, TopProducts
│  │  └─ forms/                      # ProductForm, OrderForm
│  ├─ hooks/                         # useProducts, useMoveOrder (optimista), useBulkUpdate
│  ├─ lib/
│  │  ├─ metrics.ts                  # avgTicket, grossMargin... (puras)
│  │  ├─ format.ts                   # formatCurrency, formatDate (locale/zona fijos)
│  │  ├─ filters.ts                  # lógica de filtrado (pura)
│  │  ├─ pagination.ts               # parsePagination (clamp y valores por defecto)
│  │  ├─ kanban.ts                   # columnas, transiciones permitidas, applyMove/revert
│  │  └─ schemas/                    # Zod: product, order, bulk, query
│  ├─ server/
│  │  ├─ auth.ts                     # config Auth.js
│  │  ├─ db/{client,schema,seed}.ts
│  │  ├─ services/{products,orders,analytics,kanban}.ts
│  │  └─ http.ts                     # helpers: json(), withTiming(), handleZodError()
│  └─ middleware.ts
├─ tests/                            # Vitest (unit + integración)
├─ .env.example · drizzle.config.ts · playwright.config.ts · vitest.config.ts
├─ README.md                         # URL, guía para el evaluador, cómo correr local/tests
└─ PROJECT_PLAN.md
```

---

## 8. Plan de desarrollo por fases

Cada fase termina con algo **desplegado y verificable**. No se pasa de fase con el CI en rojo.

| Fase | Contenido | Entregable verificable | Requisitos que cierra |
|---|---|---|---|
| **F0 · Cimientos** | Repo nuevo, Next.js + TS strict + Tailwind + ESLint/Prettier, shadcn base, `ci.yml` (lint, typecheck, build), proyecto en Vercel con deploy desde Actions, dominio con SSL. | URL pública con HTTPS que muestra una página; Actions en verde; un push con error de tipos **no** despliega. | RT-01, 02, 03, 08 (parcial), 09, 13 |
| **F1 · Datos + auditoría** | Neon (rama prod + rama test), esquema Drizzle, migraciones, seed determinista, `services/analytics`, `lib/metrics`, `lib/format`, `lib/pagination` y **los 3 endpoints `/api/test/*`**. Primeros unit tests. | `curl` a los 3 endpoints en producción; `Server-Timing` < 300 ms; reset repetible. | RF-60–63, RT-04, RT-11, RT-06 (parcial) |
| **F2 · Autenticación** | Auth.js con Google y GitHub, middleware, `/login`, shell de la app (sidebar, topbar, menú de usuario), apps OAuth de prod y local, pantalla de consentimiento de Google **publicada**. E2E 1 + fixture de sesión simulada + fixture de consola limpia. E2E en CI con Postgres de servicio. | Login real con ambas cuentas en producción; E2E 1 en verde en CI. | RF-01–05, RT-05, RT-07 (1/3) |
| **F3 · CRUD** | DataTable genérica (paginación de servidor, búsqueda, filtros, skeleton), productos y pedidos (listado, crear, editar, eliminar con modal), formularios con Zod, toasts. E2E 2. | Crear un producto y verlo en la tabla, también en producción; E2E 2 en verde. | RF-20–24, 30–33, RF-70, RT-07 (2/3) |
| **F4 · Kanban** | Tablero con dnd-kit, `useMoveOrder` con `onMutate` / `onError` / `onSettled`, PATCH de estado con `version`, `order_status_events`, toasts y reversión. E2E 3. | Arrastrar una tarjeta, recargar y que siga ahí; `kanban-status` refleja el cambio; un error forzado la devuelve a su columna. | RF-40–46, RF-71, 72, RT-07 (3/3) |
| **F5 · Bulk Update** | Selección de filas y barra de acciones en lote para productos y pedidos, endpoint transaccional y optimista con reversión. | Cambiar el estado de 10 pedidos a la vez; un error simulado revierte las 10 filas. | RF-50–54 |
| **F6 · Dashboard** | KPIs, gráficas (Recharts), filtro de fechas, skeletons por tarjeta, mismo servicio que `/api/test/analytics`. | Los KPIs del dashboard coinciden con `summary` del endpoint. | RF-10–14 |
| **F7 · Calidad y rendimiento** | Completar ≥ 10 unit (objetivo: 15–20), revisión de consola en todas las rutas, medición p95 de latencia en producción, revisión de accesibilidad, páginas de error. | Reporte de latencia (script con 50 peticiones); 0 errores de consola en E2E; CI en verde. | RF-73, RT-06, RT-10 |
| **F8 · Pulido y entrega** | Mejoras opcionales que dé tiempo (modo oscuro, roles, CSV...), README con guía para el evaluador, capturas, prueba en limpio de la checklist §6 desde otro equipo y otra cuenta. | Checklist §6 ejecutada por alguien ajeno al equipo. | Criterios de aceptación |

**Regla de alcance:** las mejoras opcionales (M) solo entran en F8 y solo si las fases F0–F7 están cerradas y en verde.

---

## 9. Matriz de cumplimiento

Estado actual de **todos** los requisitos: `Pendiente`. Esta tabla se actualizará al cerrar cada fase con evidencia (enlace a commit, test o captura).

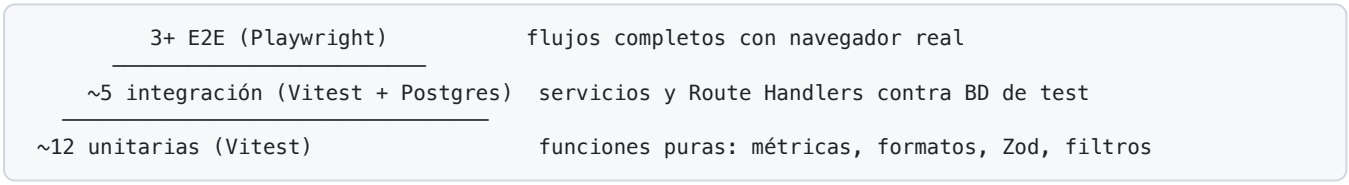
| Req. | Descripción | § | Fase | Cómo se verifica | Estado |
|---|---|---|---|---|---|
| RT-01 | Next.js ≥ 14 + App Router | 1 | F0 | `package.json`; no existe `pages/`; `next build` OK | Pendiente |
| RT-02 | TypeScript | 1 | F0 | `tsc --noEmit` en CI | Pendiente |
| RT-03 | Tailwind CSS | 1 | F0 | `tailwind` en config; sin CSS de terceros | Pendiente |
| RF-01 | OAuth Google | 3 | F2 | Login manual en producción con una cuenta Google ajena al equipo | Pendiente |
| RF-02 | OAuth GitHub | 3 | F2 | Login manual en producción con una cuenta GitHub ajena | Pendiente |
| RF-03 | Solo OAuth | 3 | F2 | Revisión de `server/auth.ts`: sin proveedor Credentials en producción | Pendiente |
| RF-04 | Rutas privadas redirigen | 5 | F2 | **E2E 1** | Pendiente |
| RF-10 | KPIs (ticket promedio, margen) | 3, 5 | F6 | Unit tests de `metrics.ts`; KPIs del dashboard = `summary` del endpoint | Pendiente |
| RF-11 / RT-04 | < 300 ms métricas y paginación | 2, 3 | F1, F7 | Cabecera `Server-Timing`; script `scripts/latency.ts` (p50/p95 de 50 peticiones en prod); assertion de `durationMs` en test de integración | Pendiente |
| RF-12 / RF-70 | Skeletons, no spinners | 4 | F3–F6 | `loading.tsx` en cada ruta + skeletons en `isPending`; búsqueda en el código sin spinners; revisión con *throttling* 3G | Pendiente |
| RF-20 | CRUD productos | 1 | F3 | Manual + tests de integración del servicio | Pendiente |
| RF-23 | Producto creado aparece en la tabla | 5 | F3 | **E2E 2** | Pendiente |
| RF-30 | CRUD pedidos | 1 | F3 | Manual + tests de integración | Pendiente |
| RF-21 / RF-32 | Paginación del servidor | 2, 3 | F3 | Red: `?page=&limit=` en peticiones; unit de `parsePagination` | Pendiente |
| RF-22 | Esquemas Zod | 5 | F1–F3 | Unit tests de esquemas (válido / inválido) | Pendiente |
| RF-40/41 | Kanban D&D por estado | 1, 3 | F4 | **E2E 3** (cambio de columna) | Pendiente |
| RF-42 | Persistencia optimista | 1, 3 | F4 | E2E 3: la tarjeta cambia de columna antes de que la petición responda (con latencia artificial en la ruta) | Pendiente |
| RF-43 / RF-72 | Reversión ante error | 4 | F4, F5 | Unit de `applyMove` / `revert`; E2E opcional con `page.route` devolviendo 500 → la tarjeta vuelve | Pendiente |
| RF-44 / RF-71 | Toast o modal de éxito/error | 4 | F4, F5 | E2E 3 comprueba el toast; manual en Bulk | Pendiente |
| RF-45 | Persiste y es consultable | 2, 6 | F4 | E2E 3 recarga la página y llama a `GET /api/test/kanban-status?orderId=` | Pendiente |
| RF-50–53 | Bulk Update atómico | 1, 3 | F5 | Unit del esquema bulk; integración: lote con un id inválido → 0 filas cambiadas | Pendiente |

| Req. | Descripción | § | Fase | Cómo se verifica | Estado |
|---|---|---|---|---|---|
| RF-60 | `GET /api/test/analytics` | 2 | F1 | `curl` en producción + test de integración (forma y paginación) | Pendiente |
| RF-61 | `POST /api/test/reset-data` | 2 | F1 | `globalSetup` de Playwright lo llama; dos ejecuciones seguidas dan los mismos conteos | Pendiente |
| RF-62 | `GET /api/test/kanban-status` | 2 | F1, F4 | `curl` antes y después de mover; E2E 3 | Pendiente |
| RT-06 | ≥ 10 unit/integración | 5 | F1–F7 | `vitest run` en CI muestra ≥ 10 pasados | Pendiente |
| RT-07 | 3 E2E | 5 | F2–F4 | `playwright test` en CI muestra 3 flujos en verde | Pendiente |
| RT-08 | `ci.yml` bloquea deploy | 5 | F0 | El job `deploy` tiene `needs:` de todos los jobs de prueba; demo: push con un test roto → no hay deploy | Pendiente |
| RT-09 | Vercel + SSL | 1, 6 | F0 | URL `https://` abre sin aviso; certificado válido (SSL Labs) | Pendiente |
| RF-73 / RT-10 | Sin errores de consola | 1 | F7 | Fixture de Playwright que **falla el test** ante `console.error` / `pageerror`; revisión manual en DevTools de cada ruta en producción | Pendiente |

---

# 10. Estrategia de pruebas

## 10.1 Pirámide




## 10.2 Pruebas unitarias e integración (mínimo 10; objetivo 15–20)

| # | Tipo | Objetivo | Casos clave |
|---|---|---|---|
| U1 | Unit | `avgTicket` | caso normal; 0 pedidos → 0 sin división entre cero; excluye cancelados |
| U2 | Unit | `grossMargin` / `grossMarginPct` | normal; ingresos 0; margen negativo |
| U3 | Unit | `formatCurrency` | centavos → `$1,234.56` MXN; negativos; 0 |
| U4 | Unit | `productSchema` | válido; precio negativo; SKU vacío; costo > precio permitido con aviso |
| U5 | Unit | `orderSchema` | sin líneas → error; cantidad 0 → error |
| U6 | Unit | `bulkUpdateSchema` | ids vacíos; más de 100 ids; cambio vacío |
| U7 | Unit | `parsePagination` | valores por defecto; `limit` > 100 se recorta o da 400; `page` no numérico |
| U8 | Unit | `filterProducts` (lógica de filtrado) | por texto, categoría, estado y combinaciones |

| # | Tipo | Objetivo | Casos clave |
|---|---|---|---|
| U9 | Unit | `kanban.applyMove` / `revertMove` | mueve entre columnas; revertir restaura el estado exacto |
| U10 | Unit | `canTransition` | transiciones permitidas y prohibidas (p. ej. desde CANCELLED) |
| I1 | Integración | `analytics.getSales` contra BD sembrada | totales conocidos del seed; paginación; `durationMs` < 300 |
| I2 | Integración | `POST /api/test/reset-data` | idempotente: mismos conteos dos veces |
| I3 | Integración | `PATCH /api/orders/:id/status` | 200 + evento registrado; 409 con versión vieja |
| I4 | Integración | `PATCH /api/products/bulk` | atómico: un id inexistente → nada cambia |
| I5 | Componente | `useMoveOrder` con QueryClient de prueba | error del servidor → la caché vuelve al estado previo |

### 10.3 Flujos E2E (Playwright)

**Preparación común:** `globalSetup` llama a `POST /api/test/reset-data`. Una fixture `authedPage` crea la **sesión simulada** firmando un JWT de Auth.js con el `AUTH_SECRET` de test y lo inyecta como cookie (no se automatiza el login real de Google, que lo impedirían el captcha y la verificación en dos pasos). Una fixture `consoleGuard` hace fallar cualquier test si aparece `console.error` o `pageerror`.

1. **Autenticación** — sin cookie: `/dashboard`, `/products` y `/kanban` redirigen a `/login`, que muestra los botones de Google y GitHub. Con la sesión simulada: `/dashboard` carga y muestra el nombre del usuario. Cerrar sesión vuelve a proteger las rutas.
2. **Creación** — `/products/new`: se llena el formulario, se envía, aparece el toast de éxito, se redirige a la tabla y se busca el SKU nuevo: la fila existe con los datos correctos. Caso inválido: precio negativo muestra el mensaje de Zod.
3. **Drag & Drop** — `/kanban`: se localiza `ORD-0001` en "Pendiente" y se arrastra a "En proceso" (con `mouse.down/move/up` por pasos, que es lo que dnd-kit necesita). Se comprueba que la tarjeta está en la nueva columna, que aparece el toast de éxito, que sigue ahí tras recargar y que `GET /api/test/kanban-status?orderId=ORD-0001` devuelve `PROCESSING`.

### 10.4 CI (`.github/workflows/ci.yml`, borrador conceptual)

```
on: push (main), pull_request
jobs:
  quality:   install (cache) → lint → typecheck → vitest (unit)
  integration: services: postgres → migrate → seed → vitest (integración)
  e2e:       services: postgres → migrate → build → next start → playwright (sube trace si falla)
  deploy:    needs: [quality, integration, e2e]; if: push a main
             vercel pull → vercel build --prod → vercel deploy --prebuilt --prod
```

- El auto-deploy de Vercel desde Git se **desactiva** para `main` (`vercel.json` → `git.deploymentEnabled`), así la **única** vía a producción es el job `deploy` que depende de las pruebas.
- Secretos de Actions: `VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`. El E2E usa `AUTH_SECRET` de prueba y credenciales OAuth ficticias (no se llama a Google ni a GitHub en CI).

---

## 11. Riesgos técnicos

| # | Riesgo | Impacto | Mitigación |
|---|---|---|---|
| R1 | **Arranque en frío** en serverless hace que la primera petición a `/api/test/analytics` supere 300 ms. | Módulo 2 (25 %) | BD y función en la **misma región**; Drizzle (cliente ligero); driver serverless de Neon con *pooling*; tabla `daily_sales`; Vercel Fluid Compute (reutiliza instancias). Evaluar el runtime Edge para ese endpoint. Medir p50/p95 en producción, no en local. |
| R2 | La latencia que mide el evaluador incluye **su red** (p. ej. México → us-east). | Lectura de la métrica | Exponer `Server-Timing` y `meta.durationMs` (tiempo del backend, que es lo que pide §3: "latencia SLA de backend"). Valorar la región más cercana a los evaluadores si Neon la ofrece en la misma zona que Vercel. |
| R3 | **Pantalla de consentimiento de Google en modo "Testing":** solo pueden entrar los usuarios de prueba y **el evaluador no podría iniciar sesión**. | Dictamen | Publicar la app ("In production"). Con los *scopes* básicos (`openid email profile`) no se requiere verificación de Google. Probar con una cuenta ajena al equipo. |
| R4 | GitHub OAuth admite **una sola URL de callback** por app. | Login roto en local o en preview | Dos OAuth Apps (local y producción); los *previews* no se usan para el login. |
| R5 | Automatizar OAuth real en E2E es inviable (captcha, 2FA). | E2E 1 | Sesión simulada con JWT firmado (el propio §5 dice "simulación de sesión"). |
| R6 | **Errores de hidratación** por fechas y moneda (zona horaria y locale distintos entre servidor y navegador). | "Sin errores de consola" | Formatear con locale `es-MX` y `timeZone` explícitos; fechas como ISO desde el servidor. |
| R7 | dnd-kit genera IDs de `aria-describedby` distintos en SSR y cliente → aviso de hidratación. | Consola | Pasar un `id` estable a `DndContext` o montar el tablero solo en cliente. |
| R8 | Recharts y `ResponsiveContainer` con ancho 0 en SSR → *warnings*. | Consola | Gráficas como componentes solo de cliente con alto fijo. |
| R9 | 404 de `favicon.ico`, imágenes de avatar bloqueadas (`next/image` sin dominio permitido) o errores de Auth.js en consola. | Consola | Favicon en `public/`; `images.remotePatterns` para `lh3.googleusercontent.com` y `avatars.githubusercontent.com`; fixture `consoleGuard`. |
| R10 | **`reset-data` público en producción**: cualquiera puede borrar los datos de demostración. | Integridad de la demo / seguridad | Ver la decisión pendiente D1. Como mínimo: solo restablece al seed (no hay datos reales), con límite de frecuencia y un flag `ENABLE_TEST_ENDPOINTS`. |
| R11 | `reset-data` tarda más que el límite de tiempo de una función de Vercel. | Endpoint caído | `TRUNCATE ... CASCADE` + inserción masiva por lotes; ~1 500 pedidos en lotes de 500; objetivo < 3 s; medido en CI. |
| R12 | Los E2E son **inestables** (flaky) por datos compartidos o animaciones del D&D. | CI en rojo intermitente | Reset antes de la suite, workers = 1 para los specs que mutan, selectores `data-testid`, esperas por estado (no `sleep`), *traces* en fallo. |
| R13 | Condición de carrera: dos usuarios mueven el mismo pedido. | Datos inconsistentes | Columna `version` → 409 → rollback y toast "el pedido cambió, recargando". |
| R14 | El dinero en `float` produce redondeos incorrectos en márgenes. | Cálculos erróneos | Enteros en centavos y redondeo solo al formatear; unit tests. |

| # | Riesgo | Impacto | Mitigación |
|---|---|---|---|
| R15 | Variables de entorno distintas entre local, CI y Vercel ( `AUTH_SECRET` , `AUTH_URL` , `DATABASE_URL` ). | Deploy roto | `.env.example` completo; validación de variables con Zod al arrancar; `AUTH_TRUST_HOST=true` . |
| R16 | Planes gratuitos: Neon suspende la BD inactiva (*scale to zero*) → la primera consulta tarda en "despertar". | R1 agravado | Revisar la configuración de *autosuspend* del plan; si no se puede desactivar, un *cron* de Vercel que la mantenga activa antes de la evaluación. Validarlo en F1. |
| R17 | Alcance: querer pulir el diseño antes de cerrar los requisitos. | Entrega incompleta | Las mejoras (M) solo entran en F8. La matriz de cumplimiento manda. |

---

## 12. Decisiones pendientes (requieren tu confirmación)

| # | Decisión | Mi recomendación |
|---|---|---|
| D1 | ¿Los endpoints `/api/test/*` son públicos o requieren sesión o token? El evaluador los llamará con `curl` o el navegador. | `analytics` y `kanban-status` : **públicos y de solo lectura**. `reset-data` : público pero con límite de frecuencia y que solo restaure el seed; documentado en el README. Si el docente lo permite, protegerlo con la cabecera `x-test-token` indicada en el README. |
| D2 | ¿Repositorio nuevo? El repo actual (SIFAT, Flask) no tiene relación. | **Repo nuevo** `omnimarket-operations-hub` ; este plan se mueve ahí. |
| D3 | ¿Postgres (Neon) o MySQL (el que ya conoce el equipo)? | Postgres en Neon. |
| D4 | ¿Cualquier cuenta Google o GitHub puede entrar? | Sí, con rol `OPERATOR` por defecto para que el evaluador pueda probar todo. Los roles más finos son una mejora opcional. |
| D5 | Moneda y zona horaria. | MXN, `es-MX` , `America/Mexico_City` . |

---

## 13. Criterios de aceptación

El proyecto se considera **terminado** cuando todo lo siguiente es cierto y está demostrado con evidencia:

1. **Producción:** la URL `https://…` en Vercel abre sin advertencias de certificado desde una red y un equipo ajenos al equipo.
2. **OAuth:** una cuenta de Google y una de GitHub **ajenas al equipo** inician sesión en producción; sin sesión, toda ruta privada redirige a `/login` ; no existe login con contraseña.
3. **Analytics:** `GET /api/test/analytics?page=1&limit=50` responde 200 con `summary` , `data` (≤ 50 filas) y `pagination` ; el tiempo de backend p95 es < 300 ms en 50 peticiones consecutivas en caliente, y la primera petición en frío está documentada.
4. **Reset:** `POST /api/test/reset-data` deja la BD con los conteos del seed `omnimarket-v1` , de forma idéntica en ejecuciones repetidas.
5. **Kanban:** al arrastrar `ORD-0001` a otra columna, la tarjeta se mueve al instante, aparece un toast, el cambio persiste al recargar y `GET /api/test/kanban-status?orderId=ORD-0001` devuelve el nuevo estado. Ante un error del servidor, la tarjeta vuelve a su columna original con un toast de error.
6. **CRUD:** productos y pedidos se pueden crear, listar (paginado), editar y eliminar; los datos inválidos se rechazan con mensajes de Zod.

7. **Bulk Update:** se pueden actualizar ≥ 2 registros a la vez en productos y en pedidos, con toast de éxito o reversión total ante error.
8. **Carga:** cada consulta o mutación asíncrona muestra un skeleton; no hay spinners genéricos.
9. **Consola:** cero `console.error` y cero excepciones no capturadas en todas las rutas (verificado automáticamente por los E2E y a mano en producción).
10. **Pruebas:** ≥ 10 pruebas unitarias o de integración y 3 E2E (autenticación, creación y Drag & Drop) pasan en CI.
11. **CI/CD:** existe `.github/workflows/ci.yml`; el último run de `main` está en verde; está demostrado que un test fallido impide el deploy a producción.
12. **Documentación:** el README incluye la URL, la guía paso a paso de la checklist del evaluador (§6), los ejemplos de `curl` de los 3 endpoints y cómo ejecutar el proyecto y los tests en local.